

Custom Shell Program

Overview

This custom shell program, written in C, provides a basic command-line interface with several advanced features. It allows users to run standard shell commands, manage background processes, handle signal interruptions, and ensures that all commands are run as child processes of the shell.

Features

1. Running Shell Commands

- The shell supports all standard shell commands including piped commands, and Input/Output redirection.
- Commands can be executed with basic functionalities except, background commands and IO redirection with piping.

2. Background Command Execution

- Commands can be executed in the background by appending an `&` at the end of the command.
- Example:

```
$ ls -l &
```

3. Background Command Queue

- The shell supports simultaneous execution of background commands.
- By default, up to 64 commands can be run in the background at the same time.
- This limit can be adjusted via command-line argument `--max-background-processes` if needed.

4. Parent-Child Command Execution Model

- The shell acts as the parent process for all commands.
- Each command is executed as a child process, ensuring isolation and better process management.
- Example:

```
$ grep "search_term" file.txt
```

In this example, `grep` runs as a child process of the shell.

5. Piped commands with inter-process communication

- Given a command with `|`(pipes) for inter process communication, the shell will split the commands, and executes each command one-by-one. The shell uses file descriptors for inter process communication.
- Each command is executed as a child process, ensuring isolation and better process management.
- Example:

```
$ echo "Hello" | wc -c
```

In this example, the shell will return answer `"8"` as character counts are 8.

6. IO Redirection

- The shell handles iutput redirections like overwrite a file with ">" and append to a file with ">>" symbols.
- The shell also handles input redirections using "<" symbol
- Example:
 - 1 Overwrite a file with ">"

```
$ echo "Hello" > test.txt
```
 - 2 Append to a file with ">>"

```
$ echo "Hello" >> test.txt
```
 - 2 Input redirection with "<"

```
$ cat < test.txt
```

7. Signal Handling for Long-running Commands

- The shell handles the CTRL+C shortcut to kill long-running commands.
- Signal handling is implemented in C to gracefully terminate the process.
- This prevents the shell itself from being terminated and allows users to stop only the current running command.

Usage

- 1 Run the shell program:

```
$ ./shell
```
- 2 List command-line argument and get help:

```
$ ./shell -h/--help
```
- 3 Change default "max-background-processes" queue size:

```
$ ./shell --max-background-processes [INT]
```
- 4 Execute a command:

```
$ ls -l
```
- 5 Execute a command in the background:

```
$ sleep 30 &
```
- 6 Terminate a long-running command with CTRL+C.

Source Code Structure

- **src/shell.c**: The main source file containing the shell implementation.
- **src/runShellCommands.h**: Header file for command execution functions.
- **src/handlePipeCommands.h**: Header file for handling inter process communication with pipes.
- **src/handleIORedirections.h**: Header file for handling IO redirections.
- **src/tokenizer.h**: Header file for command parsing and tokenization functions.

Compilation

- To compile the shell program, you can use the Makefile available in the "src" folder as follows:

- 1 **Compile the program**

```
$ make
```

- 2 **Clean compiled files**

```
$ make clean
```